Retrace: System Design Document

Version 0.3 | Based on PRD v0.5 and the decisions in section 1 | 18 October 2026 | Draft for review

This document explains how Retrace is built. Part A (high-level design) covers the architecture, tenancy, identity, communication, deployment, security, and data flows. Part B (low-level design) covers the monorepo, each service, APIs, events, jobs, algorithms, the editor, the frontend, and the build plan. The database design is a separate document that builds on this one.

Part A: High-Level Design

1. Decisions that shape the design

#	Decision	Choice	Why
1	Architecture style	Microservices: five services in v1, a sixth (Realtime) later	Independent scaling and upgrades, and clear security boundaries
2	Isolation	Frontend, backend services, and databases are separate deployables in separate network zones	Limits the damage from any single compromise
3	Tenancy	One database per tenant, plus a small central control database and a tenant registry	Strong isolation, clean deletion, and independent growth
4	Tenant database hosting	The registry maps each tenant to a server. All tenants start on one PostgreSQL server.	Low cost now, and any tenant can move to a dedicated server later
5	Data ownership	Inside each tenant database, one schema per service, and one database role per service per tenant	A service cannot read another service's tables, and a stolen credential exposes one service in one tenant

#	Decision	Choice	Why
6	Identity	Short-lived signed JWT access tokens, an HttpOnly refresh cookie, and separate service tokens between services	Every service verifies identity without a lookup
7	API frameworks	Django REST Framework used fully in Core, Ingestion, and Notifications. Django Ninja in the AI service.	Async streaming and Pydantic schemas where AI work needs them
8	Messaging	Celery for jobs, Redis Streams for cross-service events, behind an event-publisher interface, with a transactional outbox	Fewest moving parts, and the broker can be swapped later
9	Hosting	Azure Container Apps for services, a Traefik gateway container, Static Web Apps for the frontend with a custom domain and an api subdomain	Simple per-service scaling, and cookies work on one registrable domain
10	Secrets	Per-tenant data keys wrapped by a master key in Azure Key Vault	One tenant's key cannot decrypt another tenant's secrets
11	Audit log	Append-only with a hash chain	Tampering becomes detectable
12	Primary keys	UUIDv7	Unique across services, tenants, and servers, and friendly to indexes
13	Editor	v1: single-author TipTap editor storing ProseMirror JSON. Later: real-time co-editing with Yjs.	Fits the 8-week plan, with a defined path to real time
14	Repository	Monorepo with shared libraries, designed so services can be split into repositories later	Solo development

#	Decision	Choice	Why
15	Frontend toolkit	React, Tailwind CSS with shadcn/ui, TanStack Query	Fast to build

2. Target architecture and build plan

This document describes the full target architecture. Because Retrace is built by one person in about 8 weeks, the work is phased so that a thin but complete slice works by the deadline.

Phase	Weeks	Scope	Done when
1. Foundation	1 to 2	Docker Compose environment, gateway, Core (sign-up and sign-in with tokens, organizations, invitations, Admin and Member roles, tenant registry, automatic tenant database provisioning, incidents created manually or from PagerDuty, audit log), React shell	A new organization gets its own database, invites a member, and creates an incident
2. Core loop	3 to 5	Ingestion (PagerDuty, Slack, service catalog, allowlists, evidence discovery, timeline), AI service (model setting with a default model, draft pipeline with citations, blameless check), single-author editor with inline AI and comments, review and approval, PDF and Markdown export. Stretch: GitHub (deployments and merged pull requests).	An incident goes from a short description to an approved, exported postmortem

Phase	Weeks	Scope	Done when
3. Close the loop	6 to 8	Action items, Notifications (stalled reminders and email), recovery-time analytics, deployment on Azure Container Apps, demo data, tests, and polish	The full loop runs on Azure with seeded demo data

Later phases: real-time co-editing and the Realtime service, repeat-incident detection, audit-ready reports, the optional Manager and Viewer roles (Admins approve until then), full GitHub integration, richer analytics, two-factor authentication, and organization data export.

3. Architecture overview

Service	Responsibility	Framework	Data owned	Phase
Gateway	Routing, TLS, JWT verification, rate limiting, CORS, request ids	Traefik	None	1
Core	Identity and tenant management, incidents, postmortems (content, versions, comments, approvals), action items, analytics, exports, audit log	Django and DRF	Control database, and schema core in each tenant database	1
Ingestion	Integration connections, allowlists, service catalog, evidence discovery, timeline, PagerDuty webhooks	Django, DRF, Celery	Schema ingestion	2

Service	Responsibility	Framework	Data owned	Phase
AI	Model settings, draft pipeline, blameless check, inline edits, usage limits	Django Ninja (async), Celery	Schema ai	2
Notifications	Reminders, email, in-app notifications	Django, DRF, Celery beat	Schema notifications	3
Realtime	Live document sync	Django Channels	Schema realtime	Later
Web app	The user interface	React on Azure Static Web Apps	None	1

Shared infrastructure: PostgreSQL (the control database and the tenant databases), Redis (Celery brokers, Streams, cache, rate-limit counters), Azure Blob Storage (exports and evidence snapshots), Azure Key Vault (master keys and secrets), and Azure Monitor with Application Insights (logs, traces, metrics).

External system	Used for
Slack	Chat evidence, through the Web API
GitHub	Deployments, pull requests, commits, optional issue creation (stretch in v1)
PagerDuty	Incidents, alerts, notes, and webhooks
LLM provider	Drafting and rewriting, through the AI service adapter
Email provider	Invitations, password resets, reminders
Google and GitHub sign-in	User sign-in only, separate from integrations

4. Tenancy model

4.1 Control plane and tenant plane

- **Control plane (one central database, owned by Core):** users, OAuth accounts, refresh tokens, organizations, memberships, invitations, and the tenant registry. Users can belong to several organizations, so they cannot live inside one tenant database.
- **Tenant plane (one database per organization):** all business data. One organization maps to exactly one tenant database, and the organization id is the tenant id.

4.2 Tenant registry

The registry records, for each tenant: the tenant id, the database server it lives on, the database name, the schema version of each service, the status (provisioning, active, suspended, deleting), the name of the Key Vault secret for each service role, and the id of the tenant data key. Core owns the registry. Other services resolve a tenant through Core's internal API and cache the answer in Redis for a short time.

4.3 Inside a tenant database

- One schema per service: core, ingestion, ai, notifications (and realtime later).
- One database role per service per tenant. Each role can use only its own schema. For example, the AI service's role for tenant X cannot read the core schema or any other tenant's database.
- No foreign keys across schemas. Services share ids and exchange data through APIs and events.
- Each service keeps its own outbox table and migration history in its own schema.

4.4 Hosting on Azure

All tenant databases start on one Azure Database for PostgreSQL server. Because the registry holds the server for each tenant, a tenant can later be copied to a dedicated server and the registry entry updated, with no code change. Known weaknesses of the shared server are noisy neighbors, connection limits, and one outage affecting all tenants. Mitigations are a connection pooler (PgBouncer), per-tenant query timeouts, and the move-to-dedicated-server path.

4.5 Provisioning a tenant

1. A user creates an organization. Core writes the organization and a registry entry with status provisioning, and returns immediately.

2. A provisioning job picks a server, creates the database, creates the four service schemas, and creates one role per service with a random password.
3. Passwords are stored in Key Vault. A new tenant data key is generated and wrapped with the master key.
4. The job runs every service's migrations for its own schema.
5. The registry entry becomes active, and the user sees the workspace. Until then, the interface shows a short setting-up screen.
6. Any failed step is retried, and the job is safe to run again.

4.6 Schema migrations

Each service owns the migrations for its schema. At deployment, a migration runner reads the registry and applies pending migrations to every active tenant, a few at a time, recording the version per tenant and service. A tenant whose migration fails is flagged and the rollout continues for the others. Services check the version they expect, and fail clearly if a tenant is behind.

4.7 Connections

Every request carries the tenant id in its token. A shared library resolves the tenant, selects the connection for that service and tenant, and sets it for the duration of the request or job. PgBouncer pools connections so many tenant databases do not exhaust server limits.

4.8 Deleting or moving a tenant

- Deleting an organization marks it deleting, stops access, drops the database, removes its roles, Key Vault secrets, data key, and Blob files, and then removes the registry entry. This satisfies the requirement that deletion removes all data.
- Moving a tenant puts it in a read-only state, copies the database to the new server, switches the registry entry, and re-enables it.

5. Identity and access

- **Sign-in:** email and password, and OAuth with Google and GitHub, handled by Core.
- **Tokens:** Core issues a short-lived access token (10 to 15 minutes) signed with an asymmetric key. It carries the user id, the organization id, and the role. A refresh token in an HttpOnly, Secure cookie on the shared parent domain renews it. Refresh tokens rotate on use and can be revoked.

- **Verification:** the gateway and every service verify the signature using Core's published public keys, with key rotation supported. No lookup is needed per request.
- **Revocation:** a removed or demoted user keeps access until the access token expires. The short lifetime bounds this, and critical actions (changing roles, approving, deleting) also check the current role through Core.
- **Organizations:** a user in several organizations asks Core for a token for the one they are working in. All tenant routes sit under /orgs/{org_id}/.
- **Roles:** Admin and Member always exist. Manager and Viewer are optional and arrive in a later phase. Each endpoint declares allowed roles, and a shared permission class enforces them in every service.
- **Invitations:** a signed, expiring link tied to one email address and one role. Membership is invite-only.
- **Service to service:** each service gets its own service identity (a managed identity or client credentials) and presents a service token. Services accept calls only from known services, on the internal network.
- **Integration OAuth:** Slack and GitHub connections belong to the organization and are separate from user sign-in. Tokens are encrypted with the tenant data key.

6. Service communication

6.1 Synchronous calls

Services call each other's internal REST APIs for queries and commands that need an answer now (for example, the AI service asking Core to write a draft, or Notifications asking Core for stalled items). Every API publishes an OpenAPI description, which acts as the contract. Calls have timeouts and limited retries, and callers handle a service being unavailable.

6.2 Events

Services publish events to Redis Streams for things that others react to, such as incident resolved, evidence confirmed, draft ready, postmortem approved, and action item stalled. Consumers use consumer groups, so each event is handled once per service, and handlers are idempotent. All publishing goes through an event-publisher interface, so the broker can later be replaced by Azure Service Bus.

6.3 Jobs

Each service runs its own Celery workers for its own slow work, with its own queues. Jobs are never shared across services. Cross-service work happens through events or REST calls.

6.4 Reliability rules

- **Transactional outbox:** a service writes the event to its outbox table in the same transaction as the data change. A relay publishes it afterward, so an event is never lost or sent for a change that did not commit.
- **Inbox and idempotency:** consumers record handled event ids and ignore repeats.
- **Retries and dead letters:** failed events retry with backoff and then move to a dead-letter stream that Admins and developers can inspect.
- **Ordering:** events carry the tenant id, an entity id, and a version, so consumers can ignore stale updates.

7. Network and Azure deployment

Zone	Contents	Reachable from
Public	Static Web Apps (frontend) and the Traefik gateway	The internet
Application	Core, Ingestion, AI, Notifications (Container Apps with internal ingress only)	The gateway and other services
Data	PostgreSQL, Redis, Blob Storage, Key Vault, through private endpoints	Application zone only

- Only the frontend and the gateway are public. Services and data stores have no public addresses.
- The domain has two names under one registrable domain, for example app.example.com for the frontend and api.example.com for the gateway, so the refresh cookie works as a same-site cookie.
- Each service has its own managed identity with the minimum Key Vault and storage permissions.
- Container Apps scale each service independently, and idle services can scale to zero. The AI and Ingestion workers scale on queue length.

- GitHub Actions builds each changed service, runs its tests, pushes the image, and deploys. The frontend deploys to Static Web Apps. A deployment step runs the tenant migration runner.
- Environments: local Docker Compose, and Azure for the deployed system.

8. Security design

Area	Design
Tenant isolation	Database per tenant, a role per service per tenant, schemas per service, no cross-schema keys, and automated tests that try to cross tenants
Secrets	Integration tokens and model keys are encrypted with the tenant data key, which is wrapped by a Key Vault master key. The API never returns them, only a masked hint.
Identity	Short-lived signed tokens, rotating refresh tokens, rate-limited sign-in, email verification, and strong password hashing
Network	Private endpoints for data stores, internal-only service ingress, a single public gateway
Webhooks	Verify the provider signature, reject old timestamps, and keep handlers idempotent
Integration scopes	Minimum scopes (read-only where possible) plus organization allowlists as a second gate
Rich text	Sanitize stored and exported content to prevent script injection
Redaction	Optional redaction rules remove emails, tokens, and keys before storage and before any model call
Audit log	Append-only, enforced by database permissions, with each entry storing a hash of the previous entry so any change is detectable. A verify endpoint checks the chain.

Area	Design
Data deletion	Deleting an incident removes its evidence, drafts, and comments. Deleting an organization drops its database and keys.
Supply chain	Pinned dependencies, image scanning, and automated vulnerability alerts in CI

9. Main data flows

9.1 Sign-up and workspace creation

1. A user signs up (email and password, or OAuth) and verifies their email.
2. The user creates an organization and becomes its Admin.
3. Core starts tenant provisioning (section 4.5) and the interface shows progress.
4. When active, the user sets the AI model (the default is pre-selected), connects integrations, and invites members.

9.2 Incident arrives from PagerDuty

1. PagerDuty sends a signed webhook to Ingestion, which verifies it and finds the tenant from the stored integration.
2. Ingestion publishes an incident-reported event.
3. Core creates or updates the incident, and the incident appears as Resolved and waiting for a description.

9.3 Evidence discovery and timeline

1. The author submits a short description through Core, and Core publishes a description-submitted event.
2. Ingestion builds a search plan from the incident type, services, and time window, and fetches candidates from allowed sources only.
3. Each candidate is normalized, redacted, scored, and stored. The author sees the evidence preview and confirms.
4. Ingestion builds the ordered timeline with key moments and publishes timeline-ready.

9.4 Draft generation

1. The author starts generation through Core. Core publishes draft-requested, and the AI service checks the organization model setting and usage limit.
2. The AI service reads the timeline from Ingestion's API, runs the draft pipeline (section 10), and validates citations.
3. It sends the draft to Core, which stores it as a new postmortem version. The blameless check runs, and Core notifies the author.

9.5 Editing and inline AI

1. The author edits in the browser. Core stores the document as ProseMirror JSON with autosave.
2. For an inline action, the browser calls the AI service with the selected text and action. The AI service streams the suggested text back, and the author accepts or rejects it.

9.6 Review and approval

1. The author submits for review. Core checks that the blameless check has run on the latest version.
2. Reviewers comment. An Admin approves or requests changes, and the author cannot approve their own postmortem.
3. Approval stores the approver, time, and version, and publishes a postmortem-approved event.

9.7 Action items and reminders

1. Action items are created from the draft or by hand and need a confirmed owner and due date.
2. Every day, Notifications asks Core for stalled and nearly due items, sends reminders, and records each stage so none is sent twice.
3. Items that stay stalled past the escalation window also notify the Admins, or the Manager once that role exists.

9.8 Export

1. The user requests a PDF or Markdown export. Core returns a job id.
2. A Core worker renders the file, stores it in Blob Storage, and records it. The user downloads it through a short-lived signed link.

10. AI design

AI service. The AI service is built with Django Ninja. It is a good fit because inline edits are streamed (async responses and server-sent events are natural), and Pydantic schemas validate the structured output that the model must return. Ninja does not make models faster or vectors easier by itself. Model speed depends on the model endpoint and its size, and vector search comes from PostgreSQL pgvector, which works the same in any Django service.

Model adapter. All model calls go through one interface with generate (structured output) and embed (optional). Each organization has one model setting: provider, model name, endpoint, and an encrypted API key. The setting is required, and new organizations start with the Retrace default model. The default is a small, low-parameter model from the Azure education resources, so usage is limited per organization per day.

Step-by-step drafting. A small model follows long instructions poorly, so drafting is a pipeline:

1. Group timeline events into phases (detection, diagnosis, mitigation, recovery) and summarize each phase, keeping the event ids.
2. Generate each postmortem section from the phase summaries. The output must match a schema: text plus claims, each with a statement, a kind (fact or hypothesis), and supporting event ids.
3. Validate in code. Every cited id must exist, and facts need at least one id. One retry includes the error. A claim that still fails becomes a hypothesis.
4. Assemble the draft with the model name and prompt version recorded.

Blameless check. A rule-based pass finds phrases that blame or judge people. For each match, the model proposes a neutral rewrite of that one sentence, keeping all facts. The rule pass works even if the model is unavailable.

Repeat detection (later phase). It compares the same service, the root-cause category, and PostgreSQL trigram similarity of the root-cause text. Embeddings with pgvector are added only if an embedding model is configured, because a chat model setting does not guarantee an embedding endpoint.

Prompt injection. Chat messages and commit text are untrusted. Evidence is passed as clearly delimited data, the model has no tools, outputs must match a schema, and nothing is published without a human step.

Usage limits. Calls to the default model are counted per organization per day. Above the limit the AI service returns a clear message. Organizations using their own key are not limited by Retrace.

11. Reliability, scalability, and observability

- **Idempotent work.** Jobs and event handlers can run twice without harm, using unique keys.
- **Failure isolation.** If the AI service is down, users can still edit, review, and manage action items. If Ingestion is down, existing timelines still work.
- **Retries.** Provider calls use exponential backoff, respect rate-limit headers, and stop after a limited number of attempts with a clear status.
- **Partial progress.** Evidence collected before a failure is kept, and a failed draft can be retried without collecting again.
- **Timeouts.** Every external call and every job has a time limit.
- **Scaling.** Services scale independently. Workers scale on queue length. A hot tenant can be moved to a dedicated database server.
- **Observability.** OpenTelemetry traces follow a request across services using a trace id set at the gateway. Structured logs carry the tenant id, request id, and job id, never evidence text or secrets. Dashboards show latency, errors, queue depth, event lag, and model failures. Every service exposes health checks.

Part B: Low-Level Design

12. Monorepo structure

```
retrace/
  services/
    core/            Django + DRF
    ingestion/       Django + DRF + Celery
    ai/              Django Ninja + Celery
    notifications/   Django + DRF + Celery beat
    realtime/        Django Channels (later phase)
  gateway/           Traefik configuration
  web/               React app (Vite, Tailwind, shadcn/ui)
  libs/
    retrace_auth/        JWT verification, permission classes, service tokens
    retrace_tenancy/     tenant resolver, database router, context,
connection cache
    retrace_events/      event publisher, consumer, outbox relay, inbox
    retrace_common/      errors, logging, pagination, UUIDv7, crypto helpers
  contracts/         OpenAPI files and event schemas, one folder per service
  infra/             Docker Compose, Azure infrastructure files, migration
runner
  .github/workflows/ one pipeline per service, triggered by path
```

Rules that keep the services separable:

- A service never imports another service's code. It uses the other service's API or events, described in contracts.
- Shared code lives only in libs and is versioned like a package, so each service can later move to its own repository and install the libraries from a package index.
- Inside each service: apps (Django apps), api (views and serializers), services (business logic), tasks (Celery), events (publishers and handlers), tests, a Dockerfile, and an OpenAPI file.

13. API standards

13.1 Django REST Framework, used fully (Core, Ingestion, Notifications)

DRF feature	How it is used
ViewSets and routers	Resources are ModelViewSet or GenericViewSet classes registered on routers. State changes (submit, approve, reopen) are @action methods.
Serializerserializers	One serializererializer for input validation and another for output. Nested read serializerserializers for detail pages.
Authentication class	A custom class from retrace_auth verifies the JWT and sets user id, tenant id, and role on the request.
Permission classes	HasOrgRole checks the role against an allowed_roles list on each viewset or action. Object rules (for example, an author cannot approve) live in the service layer.
Throttling	Scoped throttles per user and per organization, backed by Redis. Stricter scopes for sign-in and expensive actions.
Filtering and ordering	django-filter and ordering filters on list endpoints.
Pagination	Cursor pagination for timelines and logs, page pagination for ordinary lists.
Exception handler	One handler converts typed errors into the standard error shape.
Schema	drf-spectacular produces OpenAPI. The frontend generates a typed client from it.
Versioning	URL path versioning (/api/v1).

13.2 Django Ninja (AI service)

- Routers with Pydantic schemas for requests, responses, and the structured output the model must return.
- Async views, and server-sent events for streaming inline edits.

- The same JWT verifier, role checks, error shape, and OpenAPI output as the DRF services, so the frontend sees one consistent API.

13.3 Conventions for all services

- Tenant routes sit under /api/v1/orgs/{org_id}/. A request for another tenant's object returns 404.
- JSON in and out, ISO 8601 times in UTC, UUIDv7 identifiers.
- One error shape: code, message, and optional field details.
- Slow actions return 202 with a job id, and the client reads the job status.
- Creating requests accept an Idempotency-Key header.
- Secrets are write-only. The API returns a masked hint.

14. Core service

Django apps: accounts, tenants (organizations, memberships, invitations, registry, provisioning), incidents, postmortems, actions, analytics, exports, audit, and internal (APIs used only by other services).

Data owned

Where	Tables
Control database	users, oauth_accounts, refresh_tokens, email_tokens, organizations, memberships, invitations, tenant_registry, service_credentials_refs
Tenant schema core	incidents, incident_responders, postmortems, postmortem_versions, comment_threads, comments, approvals, action_items, action_item_events, exports, audit_log, outbox, inbox

Public endpoints (through the gateway)

Area	Endpoints	Roles
Auth	POST /auth/register, /auth/login, /auth/logout, /auth/refresh, /auth/verify-email, /auth/password-reset. GET /auth/oauth/{provider}/start and /callback. GET /me.	Public, then user
Organizations	POST /orgs. GET and PATCH /orgs/{org_id}. GET and PATCH /orgs/{org_id}/settings (stall days, escalation days, retention).	Admin writes, members read
Members	GET /members, PATCH and DELETE /members/{id}, POST and DELETE /invitations, POST /invitations/accept	Admin
Incidents	GET and POST /incidents, GET and PATCH /incidents/{id}, POST /incidents/{id}/description	Members write, all read
Postmortems	GET /incidents/{id}/postmortem, POST .../generate (202), PUT /postmortems/{id} (autosave with base revision), GET .../versions, POST .../submit, .../approve, .../request-changes, .../publish, .../reopen	Members edit, Admin approves and publishes
Comments	GET and POST /postmortems/{id}/comments, PATCH /comments/{id}	All members
Action items	GET and POST /action-items, PATCH /action-items/{id}, POST /action-items/{id}/verify	Members
Analytics	GET /analytics/recovery-times, /incident-volume, /action-health	All
Exports	POST /postmortems/{id}/export (202), GET /exports/{id}/download	All
Audit	GET /audit-log, GET /audit-log/verify	Admin
Jobs	GET /jobs/{job_id}	Requester, Admin

All paths are under /api/v1/orgs/{org_id}/ except Auth and /orgs.

Internal endpoints (service tokens only): resolve a tenant, get a postmortem's text, write a generated draft as a new version, list stalled and nearly due action items, look up user display names.

Events: publishes incident.description_submitted, draft.requested, postmortem.submitted, postmortem.approved, tenant.provisioned. Consumes incident.reported, timeline.ready, draft.ready, and audit.recorded (from every service).

Jobs: provision_tenant, render_export, refresh_analytics_summaries, verify_audit_chain, purge_expired_data, expire_invitations, outbox_relay.

15. Ingestion service

Django apps: integrations (connectors for Slack, GitHub, PagerDuty), catalog, discovery, timeline, webhooks.

Data owned (schema ingestion): integration_connections, integration_secrets (encrypted with the tenant data key), allowlist_entries, services, service_links, discovery_runs, evidence_items, timeline_events, outbox, inbox.

Public endpoints

Area	Endpoints	Roles
Integrations	GET /integrations, POST /integrations/{provider}/connect, GET /integrations/{provider}/callback, DELETE /integrations/{id}, GET and PUT /integrations/{id}/allowlist	Admin
Catalog	GET, POST, PATCH, DELETE /services, PUT /services/{id}/links	Admin write, members read
Evidence	GET /incidents/{id}/evidence, PATCH /evidence/{id} (include or exclude), POST /incidents/{id}/evidence (add by link or text), POST /incidents/{id}/evidence/confirm	Members
Timeline	GET /incidents/{id}/timeline, POST, PATCH, DELETE /timeline-events/{id}	Members write, all read
Webhooks	POST /webhooks/pagerduty/{token}	Signed by PagerDuty

Internal endpoints: get the timeline and evidence for an incident (used by the AI service).

Events: consumes incident.description_submitted and publishes incident.reported, evidence.ready, timeline.ready, and audit.recorded.

Connector interface

```
@dataclass
class EvidenceCandidate:
    source: str          # slack, github, pagerduty, manual
    source_ref: str      # stable id used for deduplication
    url: str
    actor: str | None
    occurred_at: datetime
    kind: str            # message, alert, deploy, pull_request, note
    text: str

class BaseConnector:
    def fetch(self, plan: SearchPlan) -> list[EvidenceCandidate]: ...
    def check_connection(self) -> bool: ...
```

Slack limits. Slack lowered rate limits on its message-history methods for commercially distributed apps that are not in its Marketplace. Reports describe about 1 request per minute and 15 messages per request. The Slack connector therefore reads Retry-After, runs as a resumable job, reads incident and linked channels first, lets an organization use its own Slack app credentials, and accepts a Slack export upload as a fallback. Check Slack's current documentation before building it.

Jobs: discover_evidence, build_timeline, sync_pagerduty_incidents, refresh_integration_health, outbox_relay.

16. AI service

Modules: model_settings and adapter, drafting, blameless, inline_edit, usage.

Data owned (schema ai): model_settings (provider, model, endpoint, encrypted key), usage_counters, draft_runs, draft_section_runs, blameless_findings, outbox, inbox.

Public endpoints

Endpoint	Roles	Purpose
GET and PUT /ai-model	Admin	The required organization model setting
POST /ai-model/test	Admin	Connection test
POST /postmortems/{id}/sections/{key}/regenerate	Members	Regenerate one section as a suggestion
POST /postmortems/{id}/blameless-check	Members	Run the check on the latest version
GET /postmortems/{id}/blameless-suggestions	Members	List findings
PATCH /blameless-suggestions/{id}	Members	Accept, edit, or dismiss
POST /postmortems/{id}/ai/edit	Members	Inline action on a selection, streamed with server-sent events

Model adapter

```
class LLMProvider(Protocol):
    def generate(self, messages: list[dict], schema: dict, max_tokens: int) -
> dict: ...
    def embed(self, texts: list[str]) -> list[list[float]]: ...   # optional
```

A factory reads the organization setting (or the default) and returns the provider class. Providers translate errors into retryable, rate-limited, and permanent.

Events: consumes draft.requested and publishes draft.ready and audit.recorded.

Jobs: generate_draft, regenerate_section, run_blameless_check, outbox_relay.

17. Notifications service

Modules: reminders, email, in_app.

Data owned (schema notifications): reminder_log (unique per item and stage), notifications, email_log, outbox, inbox.

Endpoints: GET /notifications, PATCH /notifications/{id} (mark read), GET and PATCH /notification-settings.

Jobs: scan_stalled_items (daily), send_reminder, send_email, outbox_relay. The scan calls Core's internal API for stalled and nearly due items, so Notifications keeps no copy of action item data.

Events: consumes draft.ready, postmortem.submitted, and postmortem.approved to notify the right people.

Email goes through an adapter, so Azure Communication Services Email or any SMTP provider can be used.

18. Realtime service (later phase)

A Django Channels WebSocket service that adds real-time co-editing. When a postmortem is first opened for live editing, it converts the stored ProseMirror JSON into a Yjs document. It connects only users with a valid token, sets Viewers and approved postmortems to read-only on the server, persists updates in its own schema, and writes the editable content back to Core as snapshots on submit, approval, and publish. The editor in the web app already uses TipTap, so adding the Yjs collaboration extensions is a contained change. Until this phase, the document has one author at a time.

19. Event catalog

Events carry the tenant id, entity ids, a version, and a timestamp, never large payloads. Consumers fetch details through APIs.

Event	Published by	Consumed by	Meaning
incident.reported	Ingestion	Core	A PagerDuty incident was created or resolved
incident.description_submitted	Core	Ingestion	The author described the incident and discovery should start
evidence.ready	Ingestion	Core	Candidate evidence is ready for the preview
timeline.ready	Ingestion	Core	The confirmed timeline is built
draft.requested	Core	AI	Generate a draft
draft.ready	AI	Core, Notifications	A draft was produced and stored
postmortem.submitted	Core	Notifications	Sent for review
postmortem.approved	Core	Notifications	Approved, with the approved version
tenant.provisioned	Core	All services	A tenant is ready
audit.recorded	Every service	Core	An auditable action happened, to be appended to the audit log

20. Key algorithms

20.1 Evidence discovery (Ingestion)

Input: the incident (type, services, severity, times), the author description, the service catalog links, and the organization allowlists.

1. **Window.** Start 60 minutes before the first alert and end 30 minutes after resolution. If the incident has no PagerDuty times, the author supplies an approximate start and end. The window is shown and can be changed.
2. **Sources in priority order.** (a) PagerDuty alerts, notes, and escalations. (b) Slack channels linked to the affected services and any channel named for the incident. (c) Deployments, merged pull requests, and workflow runs for linked repositories (stretch in v1) in the window and the 2 hours before it. (d) A keyword search over other allowlisted channels using terms from the description and incident type.
3. **Security incidents.** Add terms such as login, access, token, key, permission, firewall, and suspicious, and include repository changes to access and secrets configuration.
4. **Normalize and deduplicate** by source and source_ref, then **redact** emails, tokens, and keys per the organization rules.
5. **Score** each candidate from 0 to 1 using time closeness, link to an affected service, keyword overlap with the description, same thread as a high-scoring item, authorship by a listed responder, and being a deployment shortly before the first alert.
6. **Select.** Items above the threshold are included by default. Lower items are stored as excluded so the author can add them with one click. A cap per source (for example, 300 messages) keeps drafts small. Scores and reasons are stored so the preview can explain each choice.

20.2 Timeline building (Ingestion)

1. Sort included evidence by time, then source priority, for stable ties.
2. Merge chat bursts: messages in the same thread within 2 minutes become one event.
3. Detect key moments by rules: first symptom, detection, escalation, mitigation (rollback, revert, or mitigation keywords), and resolution.
4. Split the timeline into phases at the key moments.
5. Flag events where sources disagree on time by more than a configured gap, instead of choosing silently.

20.3 Draft pipeline (AI)

1. Load the timeline by phase and summarize each phase, keeping event ids. Large phases are summarized in smaller groups first.
2. Generate each section (summary, impact, detection, response, root cause, contributing factors, what went well, what went poorly, lessons, action item suggestions) from the phase summaries. The output schema is text plus claims, each

with a statement, a kind (fact or hypothesis), and event ids.

3. Validate in code: schema shape, every cited id exists, facts have at least one id. One retry includes the error. A claim that still fails becomes a hypothesis.
4. Root cause rule: a confirmed cause is allowed only when the evidence states one explicitly (for example, a revert message naming it). Otherwise list ranked hypotheses and open questions.
5. Send the assembled draft to Core as a new version, recording the model and prompt version.

20.4 Blameless check (AI)

1. **Rule pass:** pattern groups for blame attribution (a named person as the cause), judgment (careless, sloppy), counterfactual blame (should have, failed to notice), and absolutes (always, never). A person as the subject beside a negative verb raises the match level.
2. **Rewrite pass:** for each match the model gets only that sentence and a rule (keep every fact, remove judgment, focus on the system) and returns one replacement.
3. **Review:** the author accepts, edits, or dismisses each finding. The counts are stored on the version.

20.5 Stalled items and reminders (Core data, Notifications schedule)

An item is stalled when its status is Open, In progress, or Blocked and either its due date has passed or its last update is older than the organization stall setting (default 7 days).

1. Each day Notifications asks Core for stalled and nearly due items. Core sets stalled_since when it first sees a stalled item and clears it when the item recovers.
2. Stage 0: reminder to the owner 2 days before the due date.
3. Stage 1: reminder to the owner on the first stalled day.
4. Stage 2: when stalled_since is older than the escalation setting (default 3 days), notify the Admins (and the Manager once that role exists).
5. reminder_log is unique per item and stage, so nothing is sent twice.

20.6 Repeat-incident detection (later phase)

On approval, compare the service, root-cause category, and trigram similarity of the root-cause text against approved postmortems of the same service within the lookback window (default 180 days). Score is 0.4 for a category match plus 0.6 times the text

similarity. Candidates at or above the threshold (default 0.5) become proposed links, and a person confirms or dismisses each. Repeat rate is confirmed repeats divided by incidents in the period.

21. State machines

Object	Transition	Who	Guard
Postmortem	Draft to In review	Admin, Member	Blameless check has run on the latest version and no generation is running
Postmortem	In review to Changes requested	Admin	A note is required
Postmortem	In review to Approved	Admin, not the author	The reviewed version is the latest
Postmortem	Changes requested to In review	Admin, Member	A new version exists
Postmortem	Approved to Published	Admin	None
Postmortem	Approved or Published to In review	Admin, Member	Reopen action. The document is read-only until reopened, and a snapshot is stored first.
Incident	Open to Resolved	PagerDuty event or manual	None
Incident	Resolved to Postmortem in progress	System	A draft is generated or created
Incident	Postmortem in progress to Closed	System	The postmortem is published and no critical action items are open
Action item	Open, In progress, Blocked to Done	Owner, Admin	A verification note is encouraged
Action item	Any to Dropped	Owner, Admin	A reason is required

22. Editor and frontend design

Stack. React with Vite, Tailwind CSS with shadcn/ui, React Router, TanStack Query, a typed API client generated from the OpenAPI files, and TipTap (ProseMirror) for the editor.

Editor in v1 (single author).

- The document is stored as ProseMirror JSON in Core with a revision number. Autosave sends the base revision. If it is stale (for example, a second browser tab), Core returns a conflict and the editor offers to reload.
- Citation chips are inline nodes that open the linked timeline event in a side panel.
- Comment threads are marks that carry a thread id, so they follow the text. Thread content is stored by Core.
- Selecting text opens an AI action menu (make blameless, clearer, shorter, expand with evidence, or a custom instruction). The result streams in from the AI service and shows as a visible suggestion (old text struck through, new text highlighted) that the author accepts or rejects. Suggestions are not saved until accepted.
- Approved and Published postmortems open read-only until reopened.
- To add real-time co-editing later, the Yjs collaboration extensions are added to the same editor, and the Realtime service converts documents once.

Route	Page	Roles
/login, /signup, /accept-invite	Authentication and invitation	Public
/o/:org	Dashboard: recent incidents, reviews waiting, stalled items, headline metrics	All
/o/:org/incidents	Incident list and filters	All
/o/:org/incidents/:id	Incident page with tabs: Overview, Evidence, Timeline, Postmortem, Actions, Activity	All, actions by role
/o/:org/postmortems	Library of published postmortems	All
/o/:org/actions	Action item board and stalled list	All
/o/:org/analytics	Recovery time, volume, action health	All

Route	Page	Roles
/o/:org/settings/*	Members, integrations and allowlists, service catalog, AI model, notifications, audit log	Admin

Key components: EvidencePreview (include or exclude, with the reason for each score), TimelineView (source icons, key moments, discrepancy flags), PostmortemEditor, BlamelessPanel, ApprovalBar, ActionBoard, JobProgress, OrgSwitcher, and a SettingUp screen shown during tenant provisioning.

Patterns: server state in TanStack Query; a small context for the user and current organization; a job-polling hook that starts at 2 seconds and backs off; silent token refresh on 401; role-based hiding of buttons (cosmetic, since every service enforces roles); friendly messages mapped from error codes; keyboard navigation and sufficient color contrast.

23. Tenant provisioning and migration orchestration

23.1 Database routing

A shared library (retrace_tenancy) holds a context variable for the current tenant. A Django database router reads it and returns a connection alias for that service and tenant. Connections are created on demand from the registry entry and the service role's password in Key Vault, cached per process, and closed when idle. Reading tenant data with no tenant set raises an error, so mistakes fail loudly in tests.

23.2 Provisioning job (Core)

```
provision_tenant(tenant_id):
  pick server from placement policy; record in registry
  CREATE DATABASE tenant_<id>
  for each service in [core, ingestion, ai, notifications]:
      CREATE SCHEMA <service>
      CREATE ROLE <service>_<id> with random password; GRANT usage on its
schema only
      store password in Key Vault; record secret name in registry
  generate tenant data key; wrap with Key Vault master key; store wrapped key
  run migrations for every service against the new database
  set registry status = active; publish tenant.provisioned
```

Every step checks whether it already happened, so the job can be retried safely. A failure leaves the status as provisioning with an error, and Admins see a retry action.

23.3 Migration runner

The runner is a separate container job in infra. It reads active tenants from the registry, and for each service and tenant runs that service's migrations with that service's role. It processes a few tenants in parallel, records the applied version in the registry, and continues past failures while reporting them. In a later split into repositories, each service exposes its own migrate command and the runner calls it.

23.4 Secrets and encryption

Secrets (integration tokens, model keys) use envelope encryption: a data key per tenant encrypts the secret values, and a Key Vault master key encrypts the data key. A service unwraps the data key through Key Vault using its managed identity and keeps it in memory for a short time. Rotating the master key re-wraps the data keys without re-encrypting every secret.

24. Testing, delivery, and configuration

Level	What is tested	How
Unit	Scoring, timeline building, blameless rules, stall logic, schema validation	Plain tests with fixed inputs
API	Roles, validation, state transitions for each service	Django and Ninja test clients
Tenant isolation	No read, write, or delete across tenants, and no cross-schema access by any service role	Two-tenant suite across all routes and jobs, plus direct database role tests
Contracts	Services keep to each other's OpenAPI files and event schemas	Contract tests in CI
Connectors	Slack, GitHub, PagerDuty parsing and errors	Recorded responses and a mock server
AI pipeline	Schema validation, citation checks, retries	A fake provider that returns scripted outputs, including bad ones

Level	What is tested	How
Model evaluation	Draft quality on a real model	The labeled incident package from the PRD, run against any chosen model
End to end	The core loop in a browser	A short scripted run: describe, discover, draft, review, approve, export

Delivery. GitHub Actions has one pipeline per service, triggered by path, that tests, builds an image, and deploys to Container Apps. The web app deploys to Static Web Apps. A deployment step runs the migration runner.

Configuration. Each service reads its settings from environment variables, with secrets from Key Vault. Common settings: DATABASE and registry access (through Core), REDIS_URL, KEY_VAULT_URI, JWT public key location, service identity settings, and OpenTelemetry settings. Service-specific settings: default model provider, model, endpoint, key, and daily limit (AI); Slack and GitHub app credentials and the PagerDuty webhook handling (Ingestion); email settings (Notifications); OAuth client ids for Google and GitHub and the JWT signing key (Core).

25. Build plan for about 8 weeks

Week	Deliverables
1	Monorepo, Docker Compose, gateway, shared libraries (auth, tenancy, events, common), Core accounts: sign-up, sign-in, tokens, refresh
2	Organizations, invitations, roles, tenant registry, provisioning job, migration runner, incidents, audit log with hash chain, React shell with sign-in and organization switcher
3	Ingestion: PagerDuty and Slack connections, allowlists, service catalog, webhooks, discovery job, evidence preview screen
4	Timeline builder and screen, AI service skeleton, model setting with the default model, adapter, usage limit, first draft pipeline

Week	Deliverables
5	Citation validation, blameless check, editor with comments, citation chips and inline AI, review and approval, PDF and Markdown export. Stretch: GitHub deployments and pull requests
6	Action items, Notifications service, stalled reminders, email
7	Recovery-time analytics and dashboard, Azure deployment (Container Apps, Static Web Apps, Key Vault, private networking), pipelines
8	Demo data (Kaggle seed plus generated messages and deployments), isolation and end-to-end tests, documentation, polish, and buffer

If the schedule slips, cut in this order: email delivery (keep in-app notifications), analytics, GitHub, and export formats other than PDF. Do not cut tenant isolation tests or the draft pipeline validation, since they carry the main claims of the product.

26. Risks and open items

Risk	Impact	Mitigation
Solo scope, microservices, and database per tenant together	Late or incomplete demo	Phased plan, thin slice first, shared libraries, cut order in section 25
Connection pools grow with tenants times services	Database connection exhaustion	Very small pools, PgBouncer, idle connection cleanup, move busy tenants to another server
Private networking on Azure (private endpoints, VNet integration)	Deployment time sink	Start with a simple network layout in week 7 and keep Docker Compose as the demo fallback

Risk	Impact	Mitigation
Slack history limits for distributed apps	Slow evidence discovery	Section 15: resumable connector, organization-owned Slack app option, export upload fallback
Small default model weaker than expected	Poor drafts, failed citations	Step-by-step pipeline, strict schemas, short evidence chunks, usage limits, evaluation package
JWT revocation delay	A removed user keeps access for up to 15 minutes	Short expiry, and role checks through Core on critical actions
Cookie behavior across domains	Sign-in breaks	One registrable domain with app and api subdomains
Kaggle data has no chat or code context	Weak demo timelines	Generated sample messages and deployments, marked as demo data
Tenant provisioning is slow or fails halfway	Poor first-run experience	Idempotent job, progress screen, retry action

Open items to settle: the exact default Azure model and its quota, the email provider, the custom domain name, the Kaggle dataset, the infrastructure-as-code tool (Bicep or Terraform), and how PgBouncer is arranged with per-tenant roles.

27. Changes needed in the PRD

The PRD (v0.5) was written before several decisions. These changes bring it in line with this design.

PRD item	Change
ED-1 to ED-12 and the editor risk	Real-time co-editing, live cursors, and the 10-editor target move to a later phase. v1 has a single-author editor with inline AI and comments.

PRD item	Change
REV-4, REV-7, AI-5 (version and reopen wording)	Versions are snapshots of a single-author document. Reopen remains. Regenerated text still arrives as a suggestion.
Tech stack section	Microservices, database per tenant, Azure Container Apps, Static Web Apps, Redis Streams, Django Ninja in the AI service, TipTap with ProseMirror JSON
Roles (ORG-3 and the permissions table)	Admin and Member are required. Manager and Viewer are optional and arrive in a later phase, so Admins approve in v1.
Release scope (section 15)	Replace with the 8-week phased plan. GitHub moves to a stretch goal. Repeat-incident detection and audit-ready reports move to later phases, while the audit log stays in v1.
Non-functional requirements	Add tenant database isolation tests, private network zones, per-tenant encryption keys, and the hash-chained audit log
Open questions	Replace with the open items in section 26